Instituto Tecnológico de México Campus Culiacán

Ing. En Sistemas Computacionales

Tópicos de inteligencia artificial

Problema de las N-reinas

9:00 – 10:00

Zuriel Dathan Mora Félix

Integrantes:

Ayón Zazueta Christian Natanael

Medina Ornelas Angela Ximena

**Problema de las N-Reinas**

# Búsqueda Tabú

## 1. Descripción del Problema

El problema de las N-Reinas consiste en colocar N reinas en un tablero de ajedrez N×N de forma que ninguna reina ataque a otra. Dos reinas se atacan cuando se encuentran en la misma fila, misma columna o en la misma diagonal.

Para N = 8 (caso clásico), existen 92 soluciones distintas. Conforme N crece, el espacio de búsqueda aumenta factorialmente, haciendo inviables los enfoques exhaustivos para valores grandes de N. La Búsqueda Tabú aborda este problema partiendo de un estado inicial aleatorio y mejorándolo de forma iterativa.

### Condiciones de ataque

Dos reinas en posiciones $(f_1, c_1)$ y $(f_2, c_2)$ se atacan si se cumple alguna de estas condiciones:

- Misma columna: $c_1 = c_2$
- Misma diagonal: $|c_1 - c_2| = |f_1 - f_2|$

### Objetivo

Encontrar una asignación de columnas para cada reina tal que el número de pares en conflicto sea igual a cero. La Búsqueda Tabú logra esto sin construir el árbol de búsqueda completo, trabajando directamente sobre estados completos del tablero.

## 2. Representación de P y S

### Representación del Problema (P)

| Elemento | Descripción |
|---|---|
| Tablero | N×N (N = 8 por defecto) |
| Estado inicial | Permutación aleatoria de columnas 0..N-1 |
| Restricción | Una reina por fila, garantizado por la permutación |
| Objetivo | h(estado) = 0, es decir, cero pares de reinas en conflicto |

### Representación de la Solución (S)

Una solución se representa como una lista de longitud N:

```
s = [c0, c1, ..., c_{N-1}]
```

donde $c_i$ es la columna asignada a la reina en la fila i. Usar una permutación garantiza que no existen conflictos de fila, reduciendo el espacio de búsqueda.

**Movimiento (Vecindario)**

El operador de movimiento es el intercambio (swap) de las columnas de dos filas i y j:

- Por cada par (i, j) con i < j se genera un vecino intercambiando las posiciones.
- Para N = 8: N×(N-1)/2 = 28 movimientos posibles por estado.
- Se selecciona el movimiento con menor h(s) que no esté en la lista tabú.

**Función Heurística h(s)**

Cuenta el número de pares de reinas en conflicto (misma columna o diagonal). h = 0 indica solución válida; h > 0 indica que existen conflictos y el algoritmo debe continuar buscando.

**3. Pseudocódigo del Algoritmo**

El siguiente pseudocódigo describe el funcionamiento completo de la Búsqueda Tabú, incluyendo el criterio de aspiración:

```
FUNCIÓN tabu_search(n, max_iter, tamaño_tabú):

    estado  ← permutación_aleatoria(0..n-1)
    mejor   ← estado
    lista_tabú ← cola_circular(tamaño_tabú)
    movimientos ← 0

    PARA iter = 1 .. max_iter:

        SI h(mejor) == 0:
            RETORNAR mejor

        candidatos ← []

        PARA CADA par (i, j) con i < j:
            nuevo    ← swap(estado, i, j)
            es_tabú  ← (i, j) ∈ lista_tabú
            mejora   ← h(nuevo) < h(mejor)     // criterio
de aspiración

            SI (NO es_tabú) O mejora:
```

```
                candidatos.agregar((h(nuevo), i, j, nuevo))

        Ordenar candidatos por h ascendente
        h_vec, i*, j*, estado ← candidatos[0]
        movimientos += 1

        SI h_vec < h(mejor):
            mejor ← estado

        lista_tabú.agregar((i*, j*))

    RETORNAR mejor
```

**Criterio de Aspiración**

Un movimiento tabú puede aceptarse si produce un estado con un valor h menor al mejor global conocido. Este mecanismo evita que la lista tabú bloquee caminos genuinamente prometedores.

## 4. Implementación en Python

A continuación se presenta el código completo de la implementación:

```
import time
import random
from collections import deque


def limpiar_pantalla():
    print("\n" * 40)


def imprimir_tablero(estado, n):
    separador = "+" + ("---+" * n)
    for fila in range(n):
        print(separador)
        linea = "|"
        for col in range(n):
            if estado[fila] == col:
                linea += " Q |"
            else:
                linea += "   |"
        print(linea)
    print(separador)
    print()
```

```
def heuristica(estado):
    """
    Cuenta pares de reinas en conflicto (columna o
diagonal).
    h = 0 indica solucion valida.
    """
    n = len(estado)
    conflictos = 0
    for i in range(n):
        for j in range(i + 1, n):
            if estado[i] == estado[j] or \
               abs(estado[i] - estado[j]) == abs(i - j):
                conflictos += 1
    return conflictos


def swap(estado, i, j):
    """Intercambia columnas i y j; retorna nuevo estado."""
    nuevo = estado[:]
    nuevo[i], nuevo[j] = nuevo[j], nuevo[i]
    return nuevo


def tabu_search(n, max_iter=1000, tamano_tabu=None,
verbose=True):
    """
    Busqueda Tabu para el problema de las N-Reinas.

    Parametros
    ----------
    n           : int  -- numero de reinas
    max_iter    : int  -- maximo de iteraciones
    tamano_tabu : int  -- capacidad de la lista tabu
(default: n)
    verbose     : bool -- si True, imprime el tablero en
cada paso

    Retorna
    -------
    mejor_estado : list[int]
    metricas     : dict
    """
    if tamano_tabu is None:
        tamano_tabu = n

    # Estado inicial: permutacion aleatoria
```

```
    estado = list(range(n))
    random.shuffle(estado)

    mejor = estado[:]
    h_mejor = heuristica(mejor)

    # Cola circular de tamano fijo
    lista_tabu = deque(maxlen=tamano_tabu)

    # Metricas
    inicio = time.time()
    total_movimientos = 0
    iteracion = 0

    print(f"\nBUSQUEDA TABU | N={n} | max_iter={max_iter} |
lista_tabu={tamano_tabu}")
    print(f"Estado inicial (aleatorio) | h = {h_mejor}")
    imprimir_tablero(estado, n)

    for iteracion in range(1, max_iter + 1):

        # Condicion de parada: solucion perfecta
        if h_mejor == 0:
            break

        # Generar vecindario: todos los swaps (i,j) con i <
j
        candidatos = []
        for i in range(n):
            for j in range(i + 1, n):
                nuevo = swap(estado, i, j)
                h_nuevo = heuristica(nuevo)
                movimiento = (i, j)

                es_tabu = movimiento in lista_tabu
                # Criterio de aspiracion: aceptar si mejora
el mejor global
                mejora_global = h_nuevo < h_mejor

                if not es_tabu or mejora_global:
                    candidatos.append((h_nuevo, i, j,
nuevo))

        if not candidatos:
            break

        # Seleccionar el mejor candidato (menor h)
```

```
        candidatos.sort(key=lambda x: x[0])
        h_vec, i_mov, j_mov, estado = candidatos[0]
        total_movimientos += 1

        # Actualizar mejor global
        if h_vec < h_mejor:
            mejor = estado[:]
            h_mejor = h_vec

        # Agregar movimiento a la lista tabu
        lista_tabu.append((i_mov, j_mov))

        if verbose:
            limpiar_pantalla()
            print(f"TABU | Iter {iteracion}/{max_iter} |
swap({i_mov},{j_mov})"
                  f" | h={h_vec} | h_mejor={h_mejor}")
            imprimir_tablero(estado, n)
            time.sleep(0.05)

    fin = time.time()
    tiempo_total = round(fin - inicio, 4)

    print(f"\n{chr(61)*50}")
    if h_mejor == 0:
        print(f"  SOLUCION ENCONTRADA en {iteracion}
iteraciones")
    else:
        print(f"  No se encontro solucion perfecta
(h_mejor={h_mejor})")
    print(f"{chr(61)*50}")
    imprimir_tablero(mejor, n)

    metricas = {
        "tiempo_seg"      : tiempo_total,
        "movimientos"     : total_movimientos,
        "iteraciones"     : iteracion,
        "h_final"         : h_mejor,
        "solucion_optima" : h_mejor == 0,
    }

    print(f"  Tiempo de ejecucion : {tiempo_total}
segundos")
    print(f"  Movimientos (swaps) : {total_movimientos}")
    print(f"  Iteraciones         : {iteracion}")
    print(f"  Conflictos finales  : {h_mejor}")
```

```
    print(f"  Solucion optima?    : {'SI' if h_mejor == 0
else 'NO'}")

    return mejor, metricas


def prueba_rendimiento(n, repeticiones=10):
    """
    Ejecuta la busqueda tabu varias veces y reporta
estadisticas.
    """
    print(f"\nPRUEBA DE RENDIMIENTO | N={n} |
{repeticiones} ejecuciones")

    exitos, tiempos, movimientos, iteraciones = [], [], [],
[]

    for i in range(1, repeticiones + 1):
        _, m = tabu_search(n, max_iter=2000, verbose=False)
        exitos.append(m["solucion_optima"])
        tiempos.append(m["tiempo_seg"])
        movimientos.append(m["movimientos"])
        iteraciones.append(m["iteraciones"])
        estado = "OK" if m["solucion_optima"] else "FALLO"
        print(f"  Ejecucion {i:>2}: {estado} |
{m['tiempo_seg']}s"
              f" | {m['movimientos']} swaps |
h={m['h_final']}")

    n_exitos = sum(exitos)
    print(f"\n  Tasa de exito       :
{n_exitos}/{repeticiones}")
    print(f"  Tiempo promedio     :
{round(sum(tiempos)/repeticiones, 4)}s")
    print(f"  Movimientos promedio:
{round(sum(movimientos)/repeticiones, 1)}")
    print(f"  Iteraciones promedio:
{round(sum(iteraciones)/repeticiones, 1)}")


def menu():
    print("\nPROBLEMA N-REINAS - BUSQUEDA TABU")
    n = int(input("\n  Ingrese el numero de reinas (default
8): ") or 8)

    print("\n  Opciones:")
```

```
    print("  1. Ejecutar Busqueda Tabu (con
visualizacion)")
    print("  2. Prueba de rendimiento (multiples
ejecuciones)")

    opcion = input("\n  Opcion: ").strip()

    if opcion == "1":
        max_iter = int(input("  Maximo de iteraciones
(default 1000): ") or 1000)
        tam = input(f"  Tamano lista tabu (default {n}):
").strip()
        tam = int(tam) if tam else n
        tabu_search(n, max_iter=max_iter, tamano_tabu=tam,
verbose=True)
    elif opcion == "2":
        reps = int(input("  Numero de repeticiones (default
10): ") or 10)
        prueba_rendimiento(n, repeticiones=reps)
    else:
        print("  Opcion invalida.")


if __name__ == "__main__":
    menu()
```

### 5. Métricas de Tiempo y Movimientos

El algoritmo registra las siguientes métricas en cada ejecución:

| Métrica | Variable | Descripción |
|---|---|---|
| Tiempo | tiempo_seg | Segundos desde inicio hasta fin |
| Movimientos | movimientos | Número de swaps realizados |
| Iteraciones | iteraciones | Ciclos del loop principal |
| Conflictos finales | h_final | Valor h del mejor estado encontrado |
| ¿Solución óptima? | solucion_optima | True si h_final = 0 |

### Prueba de rendimiento

La función prueba_rendimiento(n, repeticiones) ejecuta el algoritmo varias veces sin visualización y reporta tasa de éxito, tiempo promedio, movimientos promedio e iteraciones

promedio. Esto permite evaluar la robustez del algoritmo ante distintos estados iniciales aleatorios.

## 6. Evaluación y Comparativa de Algoritmos

La Búsqueda Tabú pertenece a la familia de metaheurísticas de búsqueda local. La siguiente tabla la compara con los algoritmos clásicos implementados en el proyecto:

| Algoritmo | Completo | Óptimo | Tiempo | Memoria | N grande |
|---|---|---|---|---|---|
| BFS | Sí | Sí | O(b^d) | O(b^d) alta | Inviable |
| DFS | No | No | O(b^m) | O(bm) baja | Lento |
| LDFS | No | No | O(b^l) | O(bl) baja | Depende límite |
| A* | Sí | Sí | O(b^d) | O(b^d) alta | Inviable |
| Voraz | No | No | O(b·d) | O(b) baja | Rápido/incompleto |
| Tabú | No* | No* | $O(k \cdot N^2)$ | O(N) mínima | Excelente |

* Tabú no garantiza completitud ni optimalidad, pero encuentra solución con alta probabilidad en la práctica (>95% para N=8).

### Ventajas de la Búsqueda Tabú

- Memoria eficiente: solo almacena la lista tabú de tamaño n, no el árbol completo.
- Escala bien: costo por iteración $O(N^2)$, independiente del historial de búsqueda.
- Escapa de óptimos locales mediante la prohibición de movimientos recientes.
- El criterio de aspiración permite recuperar movimientos tabú cuando producen mejoras globales.
- Alta tasa de éxito empírica: supera el 95% para N=8 en pocas iteraciones.

### Desventajas de la Búsqueda Tabú

- No garantiza encontrar la solución óptima ni completitud en todos los casos.
- Sensible a parámetros: el tamaño de la lista tabú y max_iter afectan el rendimiento.
- Requiere un estado inicial completo, no construye la solución de forma incremental.
- Puede ciclar si la lista tabú es demasiado pequeña o demasiado grande.

## 7. Aplicaciones Prácticas de la Búsqueda Tabú

La Búsqueda Tabú, propuesta por Fred Glover en 1986 y formalizada en 1989, es una de las metaheurísticas más utilizadas en optimización combinatoria. Sus aplicaciones van mucho más allá del problema de N-Reinas:

**Ruteo de vehículos (VRP)**

El Vehicle Routing Problem consiste en determinar rutas óptimas para una flota de vehículos que debe atender múltiples clientes minimizando la distancia total. Es estructuralmente idéntico al N-Reinas: un espacio de soluciones enorme (permutaciones de clientes) con una función objetivo a minimizar. Empresas como FedEx y DHL utilizan variantes de Tabú para optimizar rutas de entrega.

**Programación de horarios**

La asignación de clases a aulas, maestros y horarios en universidades es un problema de satisfacción de restricciones con millones de combinaciones posibles. La Búsqueda Tabú intercambia asignaciones (movimientos swap, análogos al N-Reinas) evitando reversiones recientes. Sistemas como UniTime utilizan Tabú para generar horarios libres de conflictos.

**Diseño de circuitos electrónicos (VLSI)**

La colocación óptima de componentes en un circuito integrado para minimizar la longitud total de cables es un problema NP-duro. Cada componente equivale a una reina; cada swap equivale a intercambiar dos componentes de posición. Herramientas de diseño como Cadence incorporan metaheurísticas tipo Tabú en sus flujos de síntesis.

**Asignación de frecuencias en telecomunicaciones**

Asignar frecuencias de radio a antenas de manera que antenas cercanas no interfieran es estructuralmente idéntico al N-Reinas: se busca una asignación sin conflictos. Tabú explora intercambios de frecuencias evitando regresar a configuraciones ya probadas. Operadores de telefonía móvil optimizan el espectro radioeléctrico con este tipo de algoritmos.

**Conexión entre la teoría y la práctica**

| Concepto en N-Reinas | Equivalente en VRP | Equivalente en VLSI |
|---|---|---|
| Estado (permutación) | Ruta de reparto | Disposición de componentes |
| Swap (i, j) | Intercambiar dos paradas | Intercambiar dos componentes |
| h(s) = conflictos | Distancia total de rutas | Longitud total de cables |

| Concepto en N-Reinas | Equivalente en VRP | Equivalente en VLSI |
|---|---|---|
| Lista tabú | Movimientos recientes prohibidos | Swaps recientes prohibidos |
| h = 0 | Costo mínimo alcanzado | Área mínima de circuito |

## 8. Referencias Bibliográficas


Glover, F. (1989). Tabu Search — Part I. ORSA Journal on Computing, 1(3), 190-206. Paper original que formaliza la Búsqueda Tabú, introduce la lista tabú y el criterio de aspiración.

Russell, S. y Norvig, P. (2021). Artificial Intelligence: A Modern Approach (4.ª ed.). Pearson. Referencia estándar en IA; cubre búsqueda local, hill climbing y metaheurísticas en el capítulo 4.

Sosic, R. y Gu, J. (1994). Efficient Local Search with Conflict Minimization: A Case Study of the N-Queens Problem. IEEE Transactions on Knowledge and Data Engineering, 6(5), 661-668.

Glover, F. y Laguna, M. (1997). Tabu Search. Kluwer Academic Publishers. Libro completo sobre Búsqueda Tabú con aplicaciones a VRP, scheduling y diseño de circuitos.

Talbi, E.-G. (2009). Metaheuristics: From Design to Implementation. John Wiley & Sons. Marco unificado para el diseño e implementación de metaheurísticas con comparativas entre Tabú, Simulated Annealing y Algoritmos Genéticos.